Convolutional Neural Networks for Sentence Classification

2023.09.14
https://arxiv.org/pdf/1408.5882.pdf

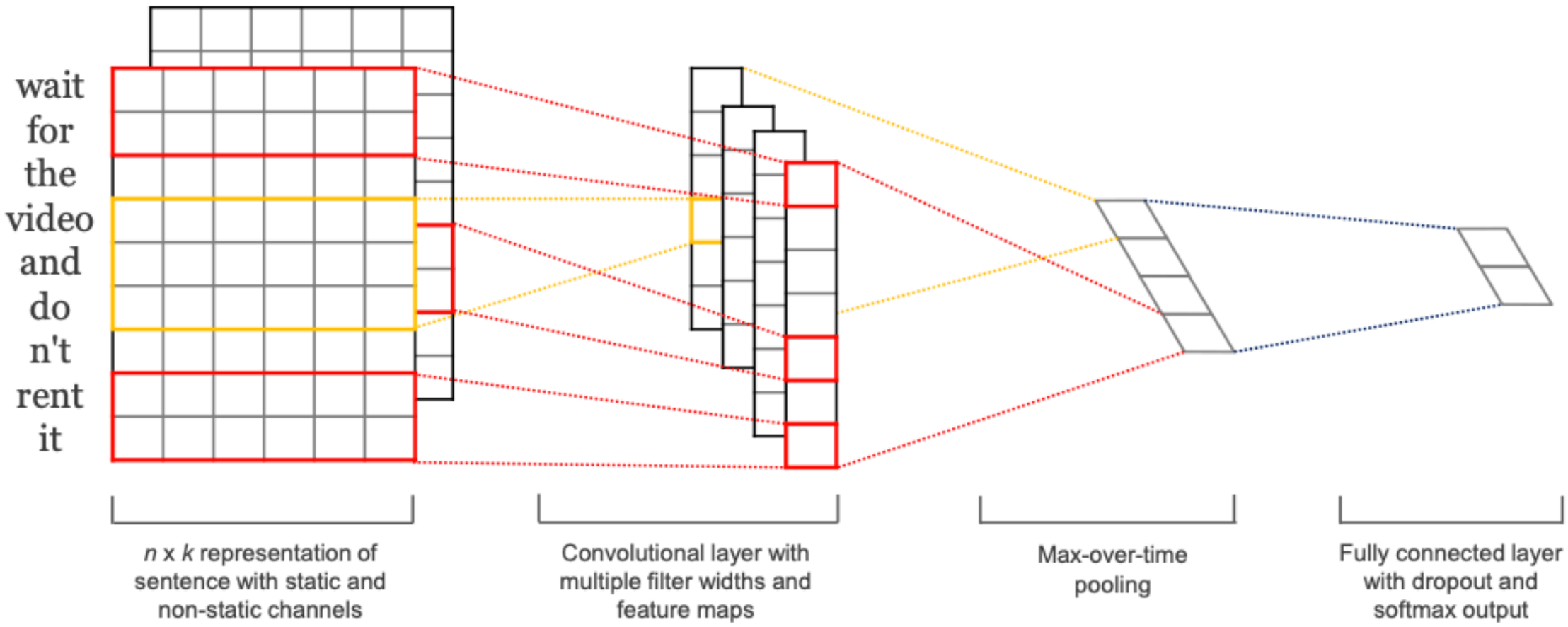
- Propose a sentence classification model using **CNN** + **word2vec**

  -> Simple cnn model + some hyperparameter tuning + static vector

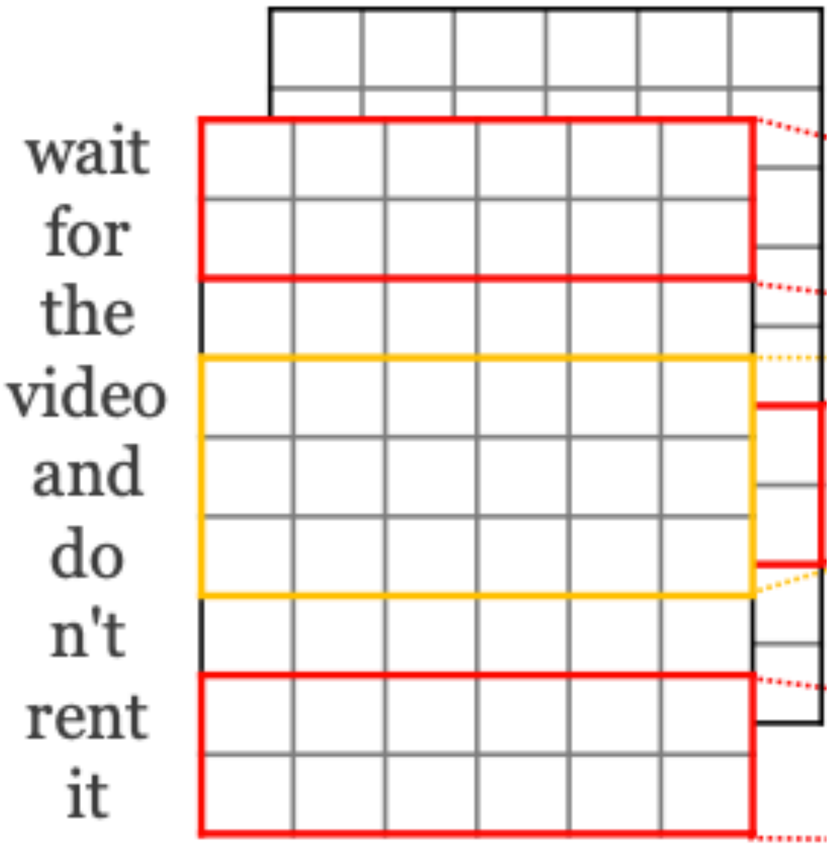
  -> With one layer on top of word vectors obtained from an unsupervised neural language model

- SOTA in **4** out of 7 tasks (sentiment classification, subjectivity …)

  -> Universal feature extractor


wait
for
the
video
and
do
n't
rent
it
n x k representation of sentence with static and non-static channels
Convolutional layer with multiple filter widths and feature maps
Max-over-time pooling
Fully connected layer with dropout and softmax output

Model



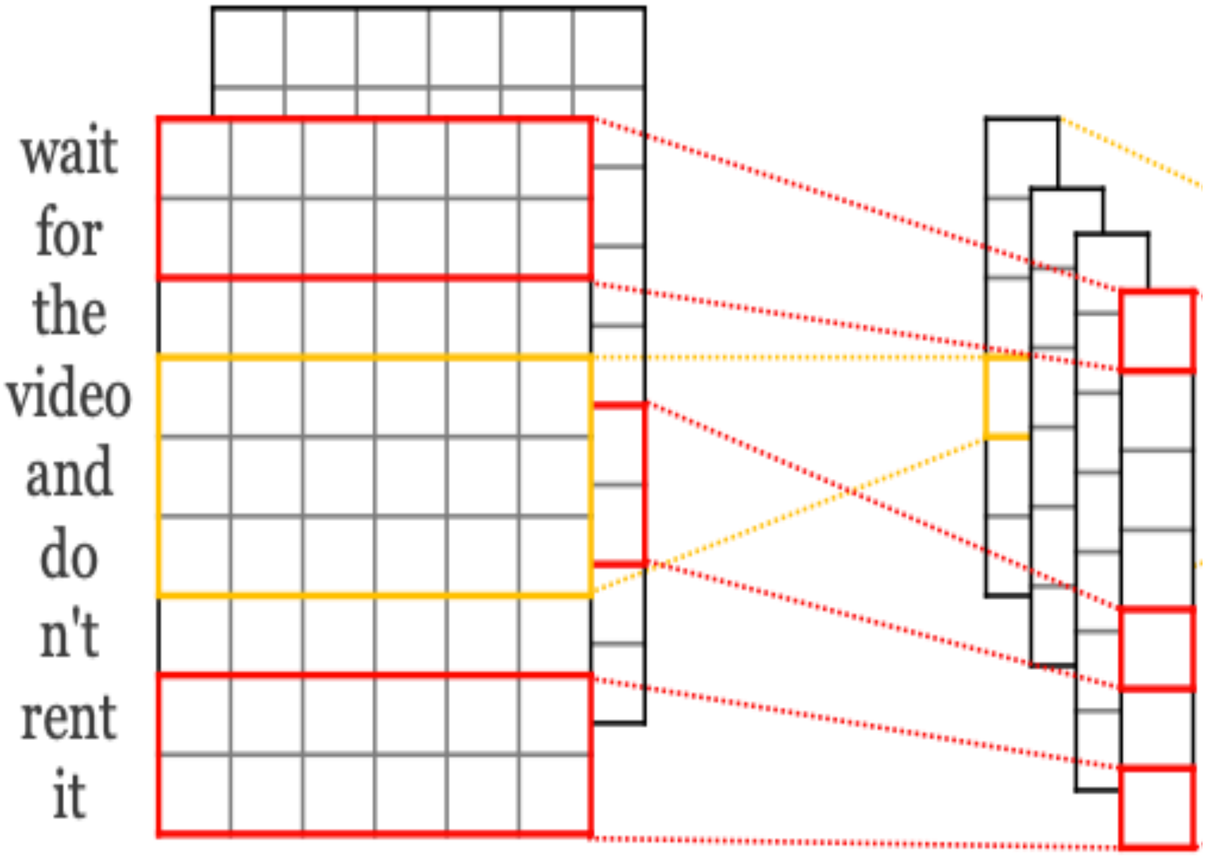
$$\mathbf{x}_i \in \mathbb{R}^k$$

k-dimensional word vector (i th)

$$\mathbf{x}_{1:n} = \mathbf{x}_1 \oplus \mathbf{x}_2 \oplus \cdots \oplus \mathbf{x}_n$$

$$\mathbf{x}_{i:i+j} = \mathbf{x}_i \oplus \mathbf{x}_{i+1} \oplus \ldots \oplus \mathbf{x}_{i+j}$$

Model



$$w \in \mathbb{R}^{hk}$$

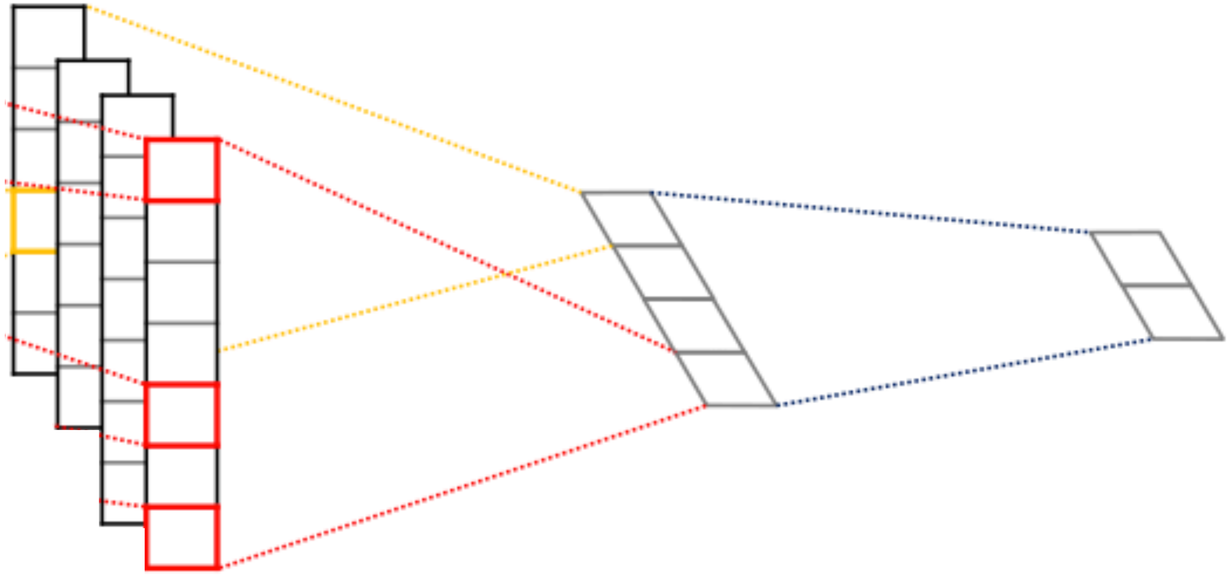
Filter size = window (h) * dimension (k)

$$\{\mathbf{x}_{1:h}, \mathbf{x}_{2:h+1}, \cdots, \mathbf{x}_{n-h+1:n}\}$$

$$c_i = f(w \cdot \mathbf{x}_{i:i+h-1} + b)$$

$$\mathbf{c} = [c_1, c_2, \cdots, c_{n-h+1}]$$

Model



$$\mathbf{c} = [c_1, c_2, \cdots, c_{n-h+1}]$$

$$\hat{c} = \max\{\mathbf{c}\}$$

Model

Fully connected layer

with dropout and

softmax output

[Dropout]

Involving only some of the weights included in the layer

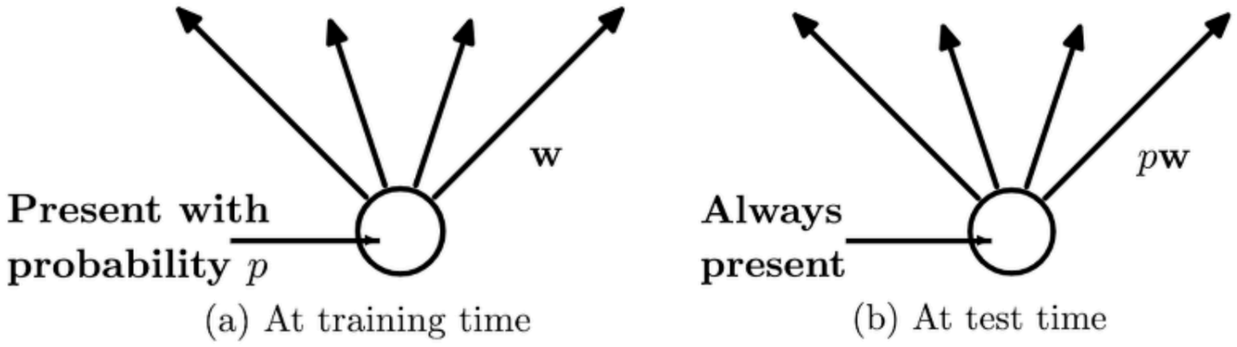
$$\mathbf{z} = [\hat{c}_1, \cdots, \hat{c}_m]$$

👉🏻 m = number of filters

$$y = \mathbf{w} \cdot (\mathbf{z} \circ \mathbf{r}) + b$$

$$\boldsymbol{r} \in \mathbb{R}^m$$

Masking vector of Bernoulli random variable with probability p



(a) At training time (b) At test time

$(\hat{\mathbf{w}} = p\mathbf{w})$

[l2 norms]

If the l2 norm of the weight vector is greater than s, rescale w to s

$$\|\boldsymbol{x}\|_2 := \sqrt{x_1^2 + \cdots + x_n^2}.$$

$$y = \mathbf{w} \cdot (\mathbf{z} \circ \mathbf{r}) + b$$

$$\mathbf{w} = \begin{cases} w, & \text{if } \|\mathbf{w}\|_2 < s \\ s, & \text{other wise} \end{cases}$$

Data	c	l	N	$\|V\|$	$\|V_{pre}\|$	*Test*
MR	2	20	10662	18765	16448	CV
SST-1	5	18	11855	17836	16262	2210
SST-2	2	19	9613	16185	14838	1821
Subj	2	23	10000	21323	17913	CV
TREC	6	10	5952	9592	9125	500
CR	2	19	3775	5340	5046	CV
MPQA	2	3	10606	6246	6083	CV

Table 1: Summary statistics for the datasets after tokenization. c: Number of target classes. l: Average sentence length. N: Dataset size. $|V|$: Vocabulary size. $|V_{pre}|$: Number of words present in the set of pre-trained word vectors. *Test*: Test set size (CV means there was no standard train/test split and thus 10-fold CV was used).

- MR : 한 문장 짜리 영화 리뷰. (긍정/부정 리뷰 판별)
- SST-1 : Stanford Sentiment Treebank – MR의 확장판, 학습/개발/테스트 분할과 세부 레이블(매우 긍정, 긍정, 중립, 부정, 매우 부정)
- SST-2 : SST-1에 중립 리뷰 제거, 이진 레이블 추가
- Subj : 주관/객관적 문장을 분류 주관성 데이터 세트
- TREC : TREC 질문 데이터 세트 — 6가지 유형(질문이 사람, 장소, 숫자 정보 등) 분류 과제
- CR : 다양한 제품(카메라, MP3 등)에 대한 고객 리뷰. 긍정/부정적 리뷰 예측 과제
- MPQA : MPQA 데이터 세트 의견 분석 서브태스크

Datasets

mini-batch size = 50 (shuffled, and randomly select 10% of training data as the dev set)

gradient check : SGD + Adadelta

filter windows (h) = [3,4,5] with 100 features maps each

activation function : ReLU

dropout rate : 0.5

L2 constraint (s) = 3

구글 뉴스에서 100억 개의 단어로 학습된 공개적으로 사용가능한 word2vec 사용

벡터 300 dim / duster bag-of-words 아키텍처 사용해 학습

Hyperparameters and Training

Model	MR	SST-1	SST-2	Subj	TREC	CR	MPQA
CNN-rand	76.1	45.0	82.7	89.6	91.2	79.8	83.4
CNN-static	81.0	45.5	86.8	93.0	92.8	84.7	**89.6**
CNN-non-static	**81.5**	48.0	87.2	93.4	93.6	84.3	89.5
CNN-multichannel	81.1	47.4	**88.1**	93.2	92.2	**85.0**	89.4
RAE (Socher et al., 2011)	77.7	43.2	82.4	–	–	–	86.4
MV-RNN (Socher et al., 2012)	79.0	44.4	82.9	–	–	–	–
RNTN (Socher et al., 2013)	–	45.7	85.4	–	–	–	–
DCNN (Kalchbrenner et al., 2014)	–	48.5	86.8	–	93.0	–	–
Paragraph-Vec (Le and Mikolov, 2014)	–	**48.7**	87.8	–	–	–	–
CCAE (Hermann and Blunsom, 2013)	77.8	–	–	–	–	–	87.2
Sent-Parser (Dong et al., 2014)	79.5	–	–	–	–	–	86.3
NBSVM (Wang and Manning, 2012)	79.4	–	–	93.2	–	81.8	86.3
MNB (Wang and Manning, 2012)	79.0	–	–	**93.6**	–	80.0	86.3
G-Dropout (Wang and Manning, 2013)	79.0	–	–	93.4	–	82.1	86.1
F-Dropout (Wang and Manning, 2013)	79.1	–	–	**93.6**	–	81.9	86.3
Tree-CRF (Nakagawa et al., 2010)	77.3	–	–	–	–	81.4	86.1
CRF-PR (Yang and Cardie, 2014)	–	–	–	–	–	82.7	–
SVM_S (Silva et al., 2011)	–	–	–	–	**95.0**	–	–

Table 2: Results of our CNN models against other methods. **RAE**: Recursive Autoencoders with pre-trained word vectors from Wikipedia (Socher et al., 2011). **MV-RNN**: Matrix-Vector Recursive Neural Network with parse trees (Socher et al., 2012). **RNTN**: Recursive Neural Tensor Network with tensor-based feature function and parse trees (Socher et al., 2013). **DCNN**: Dynamic Convolutional Neural Network with k-max pooling (Kalchbrenner et al., 2014). **Paragraph-Vec**: Logistic regression on top of paragraph vectors (Le and Mikolov, 2014). **CCAE**: Combinatorial Category Autoencoders with combinatorial category grammar operators (Hermann and Blunsom, 2013). **Sent-Parser**: Sentiment analysis-specific parser (Dong et al., 2014). **NBSVM, MNB**: Naive Bayes SVM and Multinomial Naive Bayes with uni-bigrams from Wang and Manning (2012). **G-Dropout, F-Dropout**: Gaussian Dropout and Fast Dropout from Wang and Manning (2013). **Tree-CRF**: Dependency tree with Conditional Random Fields (Nakagawa et al., 2010). **CRF-PR**: Conditional Random Fields with Posterior Regularization (Yang and Cardie, 2014). **SVM_S**: SVM with uni-bi-trigrams, wh word, head word, POS, parser, hypernyms, and 60 hand-coded rules as features from Silva et al. (2011).

- **CNN-rand**: Our baseline model where all words are randomly initialized and then modified during training.
- **CNN-static**: A model with pre-trained vectors from `word2vec`. All words—including the unknown ones that are randomly initialized—are kept static and only the other parameters of the model are learned.
- **CNN-non-static**: Same as above but the pre-trained vectors are fine-tuned for each task.
- **CNN-multichannel**: A model with two sets of word vectors. Each set of vectors is treated as a 'channel' and each filter is applied

Performance varies depending on the data

-> 항상 multichannel이 single channel보다 좋지는 않음

Model Variations

- CNN-rand : n x k의 word embedding matrix를 random한 range(-a,a)에서 초기화 및 학습

- CNN-static : n x k의 word embedding matrix를 word2vec로부터 초기화 및 해당 weight에 대해서 학습 X

- CNN-non-static : n x k의 word embedding matrix를 word2vec로부터 초기화 및 해당 weight에 대해서 학습

- CNN-multichannel : 2개의 static, non-static n x k word embedding matrix를 사용 (두가지 결과 합쳐서 진행)

Model Variations

	Most Similar Words for	
	Static Channel	Non-static Channel
bad	*good* *terrible* *horrible* *lousy*	*terrible* *horrible* *lousy* *stupid*
good	*great* *bad* *terrific* *decent*	*nice* *decent* *solid* *terrific*
n't	*os* *ca* *ireland* *wo*	*not* *never* *nothing* *neither*
!	*2,500* *entire* *jez* *changer*	*2,500* *lush* *beautiful* *terrific*
,	*decasia* *abysmally* *demise* *valiant*	*but* *dragon* *a* *and*

Table 3: Top 4 neighboring words—based on cosine similarity—for vectors in the static channel (left) and fine-tuned vectors in the non-static channel (right) from the multichannel model on the SST-2 dataset after training.

Static model, word vectors are not learned.
Non-static model, word vectors are learned

Cosine similarity between words has improved for more meaningful ones.

Randomly initialized words were learned into meaningful expressions through fine tuning.

Static vs Non-static Representations

- 다양한 필터 사이즈와 여러개 피쳐 맵을 사용할수록 모델 성능이 좋아짐

- dropout은 사용해서 2~4%의 성능 향상 보임

- pretrained word에 포함되지 않은 단어에 대해 랜덤하게 초기화할때, pretrained vector와 동일한 분포로 초기화하면 성능 향상이 있음

- Adadelta는 Adagrad와 비슷한 결과를 보였으나, 더 빨리 학습이 진행됨

https://arxiv.org/pdf/1609.04747.pdf

- Collobert가 학습한 pretrained word vector보다 word2vec이 더 좋음

Bernoulli

$$P_X(x) = \begin{cases} 1-p & (x=0) \\ p & (x=1) \end{cases}$$

$$E(x) = 0 * P_X(0) + 1 * P_X(1) = 0(1-p) + 1(p) = p$$

$$E(x^2) = 0^2 * P_X(0) + 1^2 * P_X(1) = 0(1-p) + 1(p) = p$$

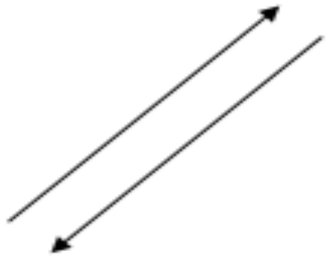
$$Var(x) = p - (p)^2 = p(1-p)$$

basic

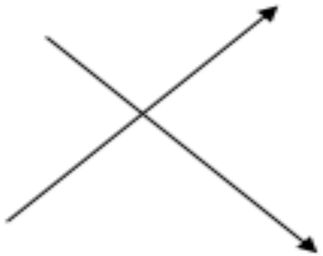
Cosine Similarity

Compare similarity between vectors

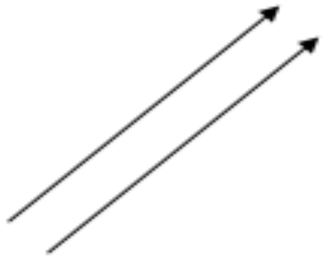
A numerical representation of how similar the angle between two vectors



코사인 유사도 : -1



코사인 유사도 : 0



코사인 유사도 : 1

$$similarity = cos(\Theta) = \frac{A \cdot B}{||A||\,||B||} = \frac{\sum_{i=1}^{n} A_i \times B_i}{\sqrt{\sum_{i=1}^{n}(A_i)^2} \times \sqrt{\sum_{i=1}^{n}(B_i)^2}}$$

https://wikidocs.net/24603

basic